

Curso Compiladores 3CT0113

Semana 3: De caracteres a tokens: Reglas, mecánica y Flex.

Semana 3

El Dilema de la Semana 2: Conflicto de Reglas

El Orden Incorrecto

```
[A-Za-z_][A-Za-z0-9]* { printf("IDENTIFICADOR: %s\n", yytext); }  
"show" { printf("PALABRA_RESERVADA: %s\n", yytext); }  
}
```

Input: "show"



Output (in an amber warning box):
"IDENTIFICADOR: show"

El Orden Correcto

```
"show" { printf("PALABRA_RESERVADA: %s\n", yytext); }  
[A-Za-z_][A-Za-z0-9]* { printf("IDENTIFICADOR: %s\n", yytext); }  
}
```

Input: "show"



Output (in a terminal green success box):
"PALABRA_RESERVADA: show"



¿Por qué Flex se confunde?

No basta con escribir una regla correcta; debemos entender la mecánica de cómo Flex elige entre reglas. Hoy decodificaremos ese motor de decisiones.

Analizador léxico

Fronteras del Analizador Léxico

1. Materia Prima (Raw Input)

`var total = 25; ----->`

2. El Filtro (Lexer)

espacios

3. El Empaque (Tokens)

[VAR]

[←]

[IDENT]

[←]

[ASSIGN]

[←]

[NUMBER]

[←]

[SEMICOLON]

4. Control de Calidad (Parser)

Lo que NO hace el Lexer:

- ✗ No comprende el significado del programa.
- ✗ No sabe si una variable ya fue declarada.
- ✗ No verifica si faltan paréntesis lógicos (sólo entrega el símbolo).

El Lexer responde: ¿Qué pieza acabo de encontrar?

¡Recordatorio!

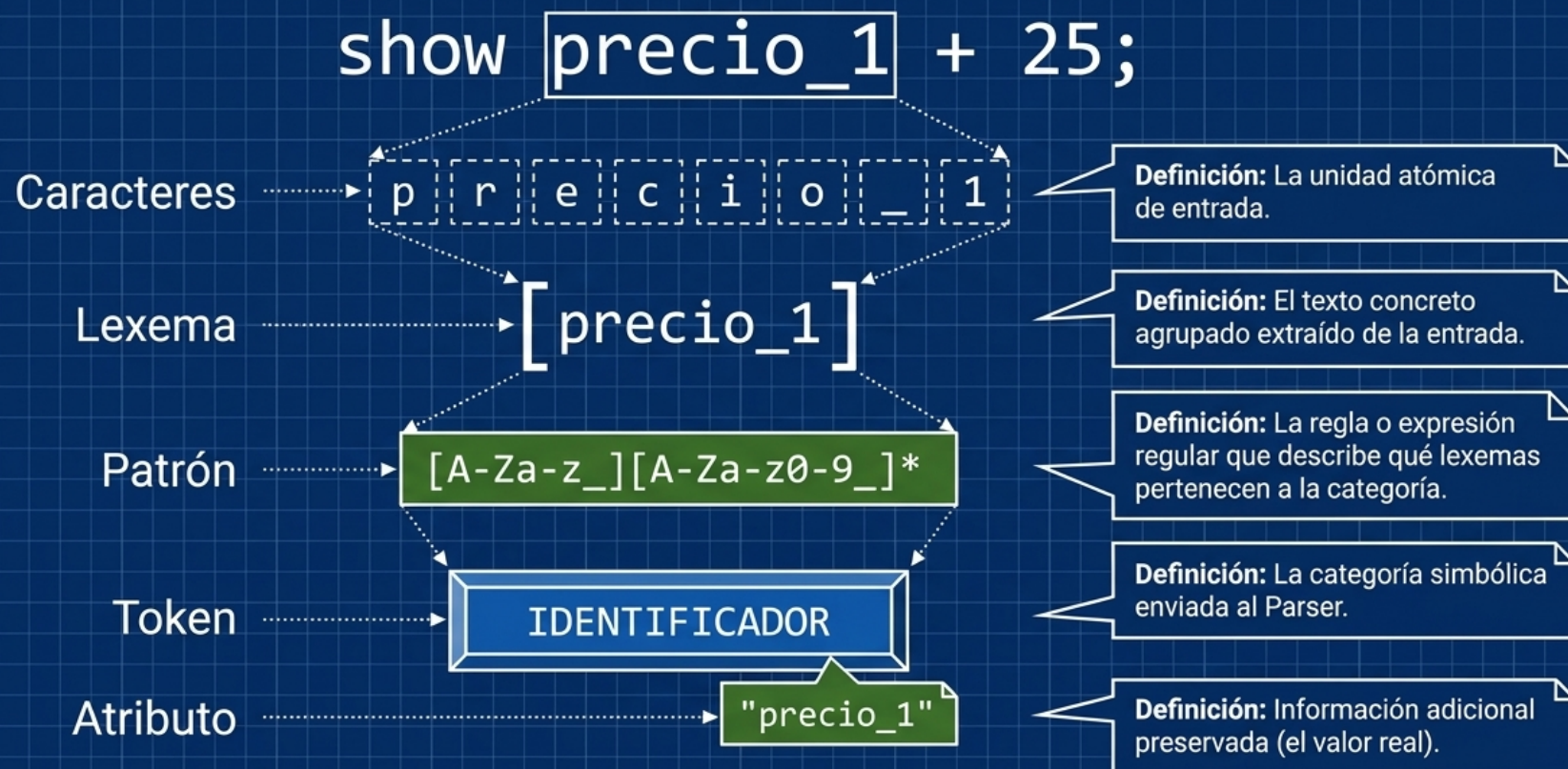
La Mesa de Triage: Clasificación de Errores

¿Quién reporta la falla?

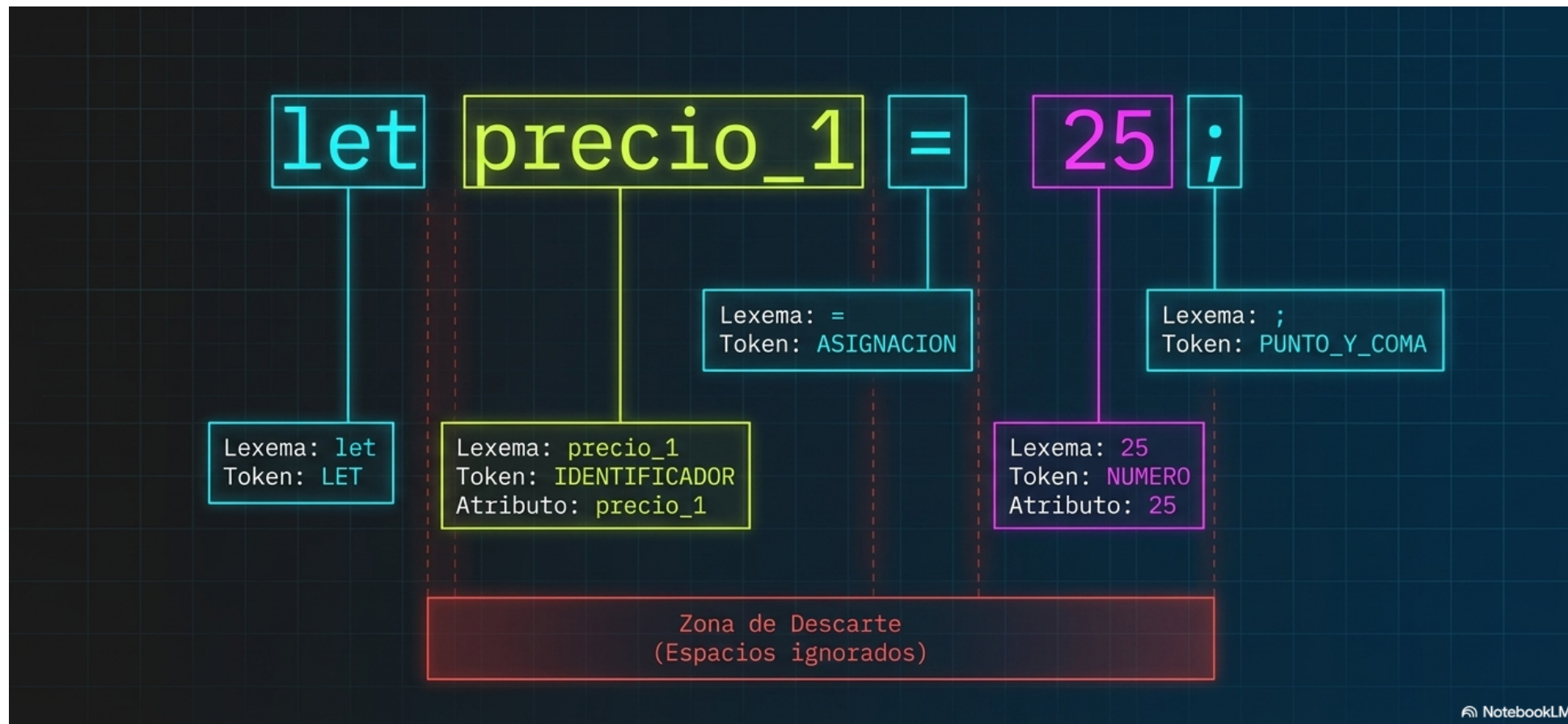
Nivel de Análisis	Entrada Defectuosa	Diagnóstico del Sistema	Responsabilidad
LÉXICO	<code>var x = 10\$;</code>	El carácter \$ no pertenece al alfabeto ni forma un patrón válido.	Lexer (Falla al crear el token).
SINTÁCTICO	<code>var = x 10;</code>	Las piezas existen, pero el orden viola la gramática esperada.	Parser (Estructura mal formada).
SEMÁNTICO	<code>show saldo;</code>	Estructura perfecta, pero saldo nunca fue declarada previamente.	Tabla de Símbolos / Analizador Semántico (Significado inválido).

Definiciones

Anatomía del Vocabulario Fundamental



Disección de una sentencia



Reglas de decisión

El Cerebro de Flex: Reglas de Decisión Absolutas

1. Máxima Coincidencia (Longest Match)

- **Flex** es glotón: siempre consumirá la mayor cantidad de **caracteres** posibles que cumplan un **patrón**.
- Un **lexema** no se cierra si el siguiente carácter aún es válido.



2. Prioridad por Orden (Desempate)

- Si dos reglas compiten y consumen exactamente la misma cantidad de caracteres...
- Gana la regla que se escribió primero (arriba) en el **archivo .l**.

Catálogo de Tokens de nuestro ejemplo

Catálogo de Tokens: Especificaciones de nuestro ejemplo

Categoría	Token (Salida Flex)	Patrón Flex (Regex)	Ejemplos Aceptados	Acción / Comportamiento
Palabras Reservadas	INPUT, SHOW, VAR, END	"input", "show", etc.	var, show	Imprimir Token Exacto
Identificadores	IDENT	[A-Za-z_][A- Za-z0-9_]*	precio_1, x	Imprimir IDENT(yytext)
Números	NUMBER	[0-9]+	25, 2026	Imprimir NUMBER(yytext)
Operadores & Signos	ASSIGN, PLUS, SEMICOLON	"=", "+", ";"	=, +, ;	Imprimir Símbolo Exacto
Espacios (Ignorados)	(Ninguno)	[\t\r\n]+	(Tabs, Saltos)	ACCIÓN VACÍA (Consumir)
Seguridad (Error)	ERROR_LEXICO	." (Punto final)	\$, @, #	Reportar Error + yytext

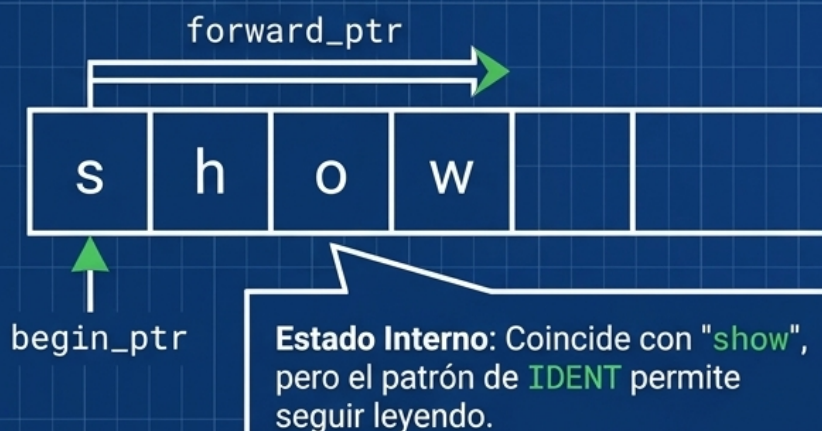
Lookahead

Mecánica del Scanner: El Sliding Window (Lookahead)

¿Cómo sabe el Lexer cuándo termina una palabra?

s	h	o	w	c	a	s	e	=	2	5	;
---	---	---	---	---	---	---	---	---	---	---	---

Paso 1: Inicio de Escaneo



Paso 2: Lookahead (Mirar hacia adelante)



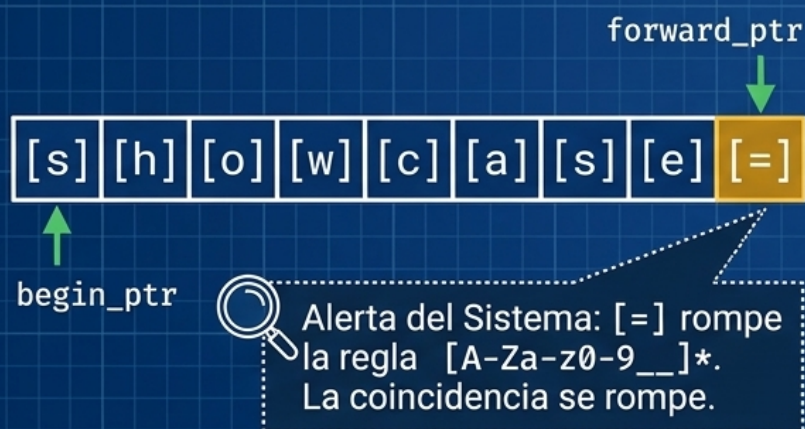
Generación de token

Mecánica del Scanner: Retracción y Cierre

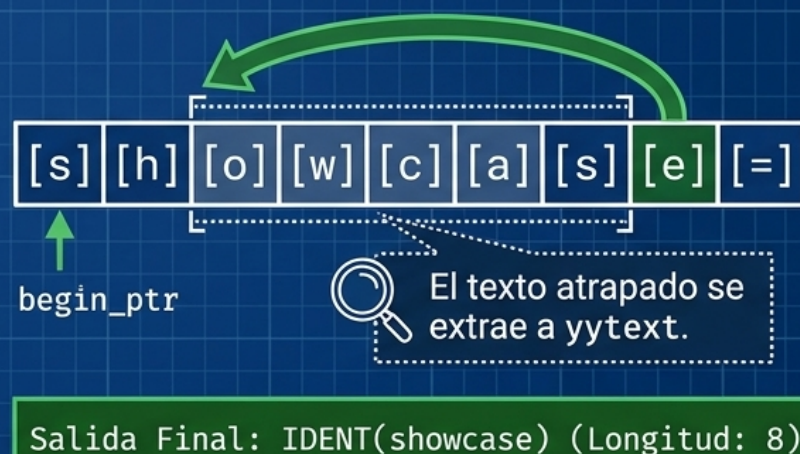
El momento en que se produce el Token

[s]	[h]	[o]	[w]	[c]	[a]	[s]	[e]	[=]	[2]	[5]	[;]
-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

Paso 3: El Quiebre (Mismatch)



Paso 4: Retracción y Producción

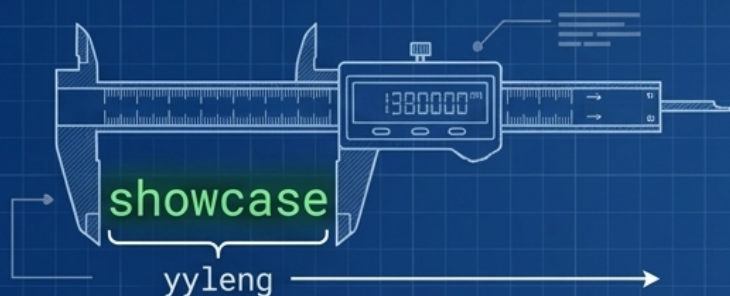


Recordatorio de reglas de decisión

El Motor de Decisiones de Flex

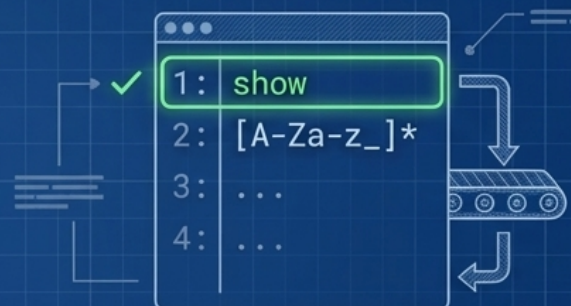
¿Qué pasa cuando varias reglas aplican al mismo texto?

Regla 1: Máxima Coincidencia (Longest Match)



- Flex siempre elegirá la regla que consuma la mayor cantidad de caracteres de la entrada (yyleng más alto).
- **Justificación:** Evita que una palabra como showcase se fragmente prematuramente en show y case.

Regla 2: Prioridad por Orden de Aparición



- Si dos reglas empatan consumiendo exactamente la misma cantidad de caracteres, gana la regla escrita primero (más arriba) en el archivo .l.
- **Justificación:** Permite definir palabras clave específicas ("show") antes del patrón genérico de identificadores ([A-Za-z_]*).

Matriz de predicción

Matriz de Predicción: "show" vs IDENT

Reglas en competencia: 1) "show" 2) [A-Za-z_][A-Za-z0-9_]*

Cadena de Entrada

Análisis de Longitud
(Lookahead)

Regla Ganadora

Salida Final

show

Empate a 4 caracteres
(show vs IDENT).

Regla 2
(Orden)

PALABRA_RESERVADA

showcase

"show"=4 chars.
IDENT=8 chars.

Regla 1
(Más Larga)

IDENT(showcase)

show x

Espacio rompe el
lexema en show.
Empate a 4.

Regla 2
(Orden)

PALABRA_RESERVADA
+ (espacio) +
IDENT(x)

show1

"show" falla en el 1.
IDENT=5 chars.

Regla 1
(Más Larga)

IDENT(show1)

Archivo Flex

Anatomía Detallada del Archivo Flex (.l)

Tres secciones, un solo esquema arquitectónico

1. Sección de Definiciones

```
%option noyywrap
```

Evita enlazar librerías externas para este escáner básico.

```
%{
```

```
#include <stdio.h>
```

```
%}
```

Código C puro que se inyecta directamente al inicio.

..... %% (Separador Estructural)

2. Sección de Reglas

```
"show" { printf("SHOW\n"); }
```

```
[0-9]+ { printf("NUMBER(%s)\n", yytext); }
```

El corazón lógico.
Izquierda = Patrón,
Derecha = Acción C.

..... %% (Separador Estructural)

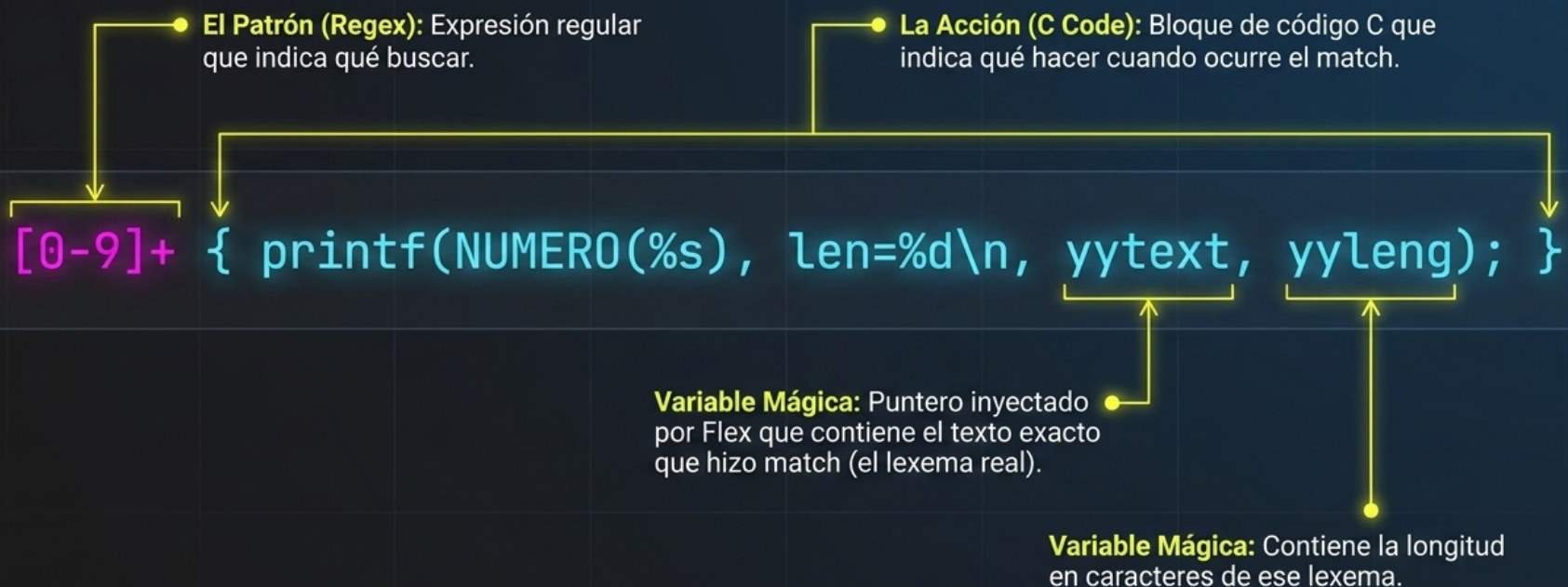
3. Código de Usuario

```
int main(void) { return yylex(); }
```

Dispara el motor de reconocimiento.

Patrón y acción

El Corazón de las Reglas: Patrón y Acción



Ignorar datos y errores léxicos

Datos Ignorados y la Red de Seguridad

Consumo Silencioso (Whitespace)

```
[ \t\r\n]+ ;
```

► Mecánica:

Reconoce espacios, tabuladores y saltos de línea, pero la acción está vacía (punto y coma).

► Impacto:

Flex avanza sus punteros, consumiendo la basura, sin enviar tokens innecesarios al Parser.



La Red de Seguridad (El Punto Final)

```
. { printf("ERROR_LEXICO: %s\n", yytext); }
```

► Mecánica:

El punto . en regex significa "cualquier carácter". Como está al final del archivo, solo se activa si todas las reglas anteriores fallan (Regla 2: Orden).

► Impacto:

Evita que el escáner colapse o trague caracteres inválidos silenciosamente (Ej: var x = 10\$; atrapará el \$).



Scanner en Java

← → ↻

craftinginterpreters.com/scanning.html#top

CRAFTING
INTERPRETERS

Scanning 4

The Interpreter Framework 4.1

Lexemes and Tokens 4.2

Regular Languages and Expressions 4.3

The Scanner Class 4.4

Recognizing Lexemes 4.5

Longer Lexemes 4.6

Reserved Words and Identifiers 4.7

CHALLENGES

IMPLICIT SEMICOLONS note

← PREVIOUS

↑ UP

NEXT →

```
private boolean isAtEnd() {
    return current >= source.length();
}
```

```
<< lox/Scanner.java
add after scanTokens()
```

Recognizing Lexemes 4.5

In each turn of the loop, we scan a single token. This is the real heart of the scanner. We'll start simple. Imagine if every lexeme were only a single character long. All you would need to do is consume the next character and pick a token type for it. Several lexemes *are* only a single character in Lox, so let's start with those.

```
private void scanToken() {
    char c = advance();
    switch (c) {
        case '(': addToken(LEFT_PAREN); break;
        case ')': addToken(RIGHT_PAREN); break;
        case '{': addToken(LEFT_BRACE); break;
        case '}': addToken(RIGHT_BRACE); break;
        case ',': addToken(COMMA); break;
        case '.': addToken(DOT); break;
        case '-': addToken(MINUS); break;
        case '+': addToken(PLUS); break;
        case ';': addToken(SEMICOLON); break;
        case '*': addToken(STAR); break;
    }
}
```

```
<< lox/Scanner.java
add after scanTokens()
```

Wondering why / isn't in here? Don't worry, we'll get to it.

<https://craftinginterpreters.com/scanning.html>

UNIVERSIDAD
SAN
PABLO
GUATEMALA

¡Gracias!